PySwelFlo User Manual

1. Introduction

PySwelFlo is a Python companion toolkit for automating workflows around SwelFlo steady-state geothermal wellbore simulations. It is intended for reservoir engineers who want to use Python to prepare simulation inputs, organize multiple cases, launch supported SwelFlo runs, and process simulation results.

PySwelFlo does not contain the SwelFlo simulation engine. Users must have access to a valid SwelFlo installation and must comply with its licensing terms.

2. Scope and workflow

A typical workflow is:

- Start from an existing, validated SwelFlo model.
- Read the model using `SWFinput`.
- Inspect or modify the wellhead conditions, casing/liner geometry, reservoir temperature points, and feedzones.
- Write a separate working directory for each simulation case.
- Run the model using the compatible executable mode, or open it in the SwelFlo GUI.
- Read `SWFoutput###.txt` and any associated CSV files using `SWFoutput`.
- Analyze the resulting pandas DataFrames and export selected data to Excel or JSON.

Recommended practice: preserve the original SwelFlo project and create a separate directory for each automated case. This avoids accidental overwriting and makes the workflow reproducible.

3. Installation

3.1 Requirements

The current source imports the following third-party packages:

- NumPy
- pandas
- Matplotlib
- Plotly
- pyXSteam

Install them with:

```
python -m pip install numpy pandas matplotlib plotly pyXSteam
```

Install PySwelFlo from the repository root:

```
python -m pip install -e .
```

The repository metadata currently declares Python `>=3.10`. Because dependency versions and syntax compatibility can evolve, validate the installation in the Python environment used for your engineering workflow.

3.2 Project layout and import

PySwelFlo currently uses a single-module layout:

```
PySwelFlo/
■■■ pyproject.toml
■■■ README.md
■■■ src/
■   ■■■ pyswelflo.py
■■■ docs/
■■■ examples/
```

The `pyproject.toml` configures setuptools to install `pyswelflo.py` as the top-level `pyswelflo` module. Users should install the project from its repository root rather than adding the `src` directory manually to `PYTHONPATH`.

After installation, import the classes as follows:

```
from pyswelflo import SWFinput, SWFoutput, Feedzone
```

For a development checkout, install in editable mode with `python -m pip install -e .`. This keeps the import interface the same as a regular installation.

4. Input model handling: `SWFinput`

4.1 Load an existing model

```
from pyswelflo import SWFinput

model = SWFinput(r"C:\SwelFlo\Project\SWFinput.txt")
model.read()

print(model)
```

`SWFinput(filepath=None)` creates an input-model object. If no path is supplied, the current implementation derives a default input path from its configured engine directory. For most user workflows, explicitly provide the path to a known SwelFlo input file.

`read()` reads the file into the object. It also looks for a sibling `SWFunitsChoices.txt` and reads unit selections when that file exists.

4.2 Input data represented by the package

```
| Class | Stored fields |
| `WellheadCond` | `P`, `Q`, `T`, `H`, `Ho`, `X`, `Xo`, `XCO2` |
| `CasingSection` | `ele`, `secl`, `dev`, `trued`, `casing_choice`, `rad`, `eps`, `delz` |
| `ReservoirTemperature` | `depth`, `temp` |
| `Feedzone` | `depth`, `ficode`, `pressure`, `temperature`, `enthalpy`, `x`, `xCO2`, `values` |
```

These are the field names used by the current Python source. Refer to the SwelFlo input specification and a validated model to confirm physical meaning, units, and allowed ranges before editing values.

4.3 Inspect input data

```
tables = model.to_dataframe()

wellhead_df = tables["wellhead_conditions"]
casing_df = tables["casing_sections"]
reservoir_temperature_df = tables["reservoir_temps"]
feedzone_df = tables["feedzones"]

print(wellhead_df)
print(casing_df)
```

`to_dataframe()` returns a dictionary of pandas DataFrames for the wellhead conditions, casing sections, reservoir temperatures, and feedzones.

`to_dict()` returns a dictionary representation of the main input-model fields.

4.4 Add and update model components

The current `SWFinput` API includes:

```
model.add_wellhead_condition(P, Q, T, H, Ho, X, Xo, XCO2)
model.update_wellhead_condition(index, **kwargs)

model.add_casing_section(ele, secl, dev, trued,
                         casing_choice, rad, eps, delz)
model.update_casing_section(index, **kwargs)

model.add_reservoir_temp(depth, temp)
model.update_reservoir_temp(index, **kwargs)

model.add_feedzone(depth, ficode, pressure, temperature,
                   enthalpy, x, xCO2, values)
model.update_feedzone(index, **kwargs)
```

Indices are zero-based Python indices. Update methods silently do nothing when the index is outside the corresponding list. The feedzone add/update methods check that the feedzone depth does not exceed `model.length`; calculate or set the well length appropriately before adding feedzones.

Example:

```
# Example only: use values and units from your validated model.
model.update_wellhead_condition(0, P=YOUR_VALUE)
```

Replace `YOUR_VALUE` with a numeric value appropriate to the actual SwelFlo field and unit convention. Do not copy placeholder values into a production model.

4.5 Casing length

`calc_length()` sums the `secl` values in `casing_sections` and stores the result in `model.length`.

```
model.calc_length()
print(model.length)
```

Use this method after loading or changing casing sections if the total well length needs to be recalculated.

4.6 Write an input model

```
model.write(r"C:\SwelFlo\Runs\Well_A_case_01")
```

The method creates the output directory if necessary and writes:

- `SWFinput.txt`
- `SWFunitsChoices.txt`

The current implementation preserves raw reservoir-temperature rows and the trailing Monte-Carlo/Batch block when reading and rewriting a model. This is intended to retain file sections that may be required by SwelFlo but are not fully represented by the Python data model.

When building a model from scratch, verify that every required SwelFlo input section is populated. A newly initialized object is not necessarily a complete runnable SwelFlo model.

4.7 Casing visualization

```
fig = model.plot_casing(show_feedzone=True)
fig.show()
```

`plot_casing()` returns a Plotly figure. It can display casing/liner geometry and optionally feedzone depths. Its optional arguments include `max_x`, `normalize`, `scale`, `fig`, `title`, and `show_feedzone`.

5. Running SwelFlo

5.1 Silent execution

```
model.run(
    swelflo_path=r"C:\Path\To\SwelFlo\swelflo.exe",
    output_dir=r"C:\SwelFlo\Runs\Well_A_case_01"
)
```

The current implementation invokes:

```
swelflo.exe -s -f SWFinput.txt -p <output_directory>
```

It writes `SWFunitsChoices.txt` into the working directory before execution and checks whether a new `SWFoutput...` file was produced.

Compatibility warning: source comments document that a tested older 2017 build supports silent execution, while a tested V9.105 build can crash in silent mode. Do not assume all SwelFlo versions support the same command-line behavior.

5.2 GUI execution

```
model.run_gui(
    swelflo_path=r"C:\Path\To\SwelFlo\swelflo.exe",
    output_dir=r"C:\SwelFlo\Runs\Well_A_case_01"
)
```

This launches SwelFlo without the silent flag, with the input and output directory supplied to the executable. The call returns the command string; the user must run the simulation in the GUI.

6. Output processing: `SWFoutput`

6.1 Load a result

```
from pyswelflo import SWFoutput

result = SWFoutput(
    r"C:\SwelFlo\Runs\Well_A_case_01\SWFoutput001.txt"
)

print(result)
```

The constructor reads the result text file, rejects missing or empty files, and extracts available sections and tables.

6.2 Extracted sections and tables

The current implementation builds the following keys when parsing:

Sections

- `setup_summary`
- `downhole_profile`
- `summary_stats`

DataFrames

- `casing_geometry`
- `all_wellhead_blocks`
- `wellhead_info_summary`
- `feedzone_results`

- `wellbore_profile`

When matching sibling files are present, it may also load:

- `plot_profile` from `SWFplot###.csv`
- `output_curve` from `SWFoutcrv###.csv`

Some tables can be empty if a section is absent or does not match the parser's expected format. Always inspect `result.dataframes.keys()` and the DataFrame contents before using them in calculations.

```
print(result.sections.keys())
print(result.dataframes.keys())

profile = result.dataframes["wellbore_profile"]
print(profile.head())
```

6.3 Sibling CSV files

The parser looks for CSV files beside the output text file. The expected naming patterns are:

- `SWFplot###.csv`
- `SWFoutcrv###.csv`

The CSV readers are also available as static methods:

```
plot_df = SWFoutput.read_plot_csv(r"path\SWFplot001.csv")
curve_df = SWFoutput.read_outcrv_csv(r"path\SWFoutcrv001.csv")
```

The source docstrings describe the plot CSV as containing depth, simulated pressure, simulated temperature, and velocity; and the output-curve CSV as containing simulated flow, wellhead pressure, and flowing enthalpy. Confirm exact column labels against the CSV produced by your SwelFlo build.

6.4 Export results

Export all DataFrame tables to an Excel workbook:

```
result.to_excel("Well_A_results.xlsx")
```

Obtain a Python dictionary suitable for JSON serialization:

```
data = result.to_json()
```

`to_json()` returns a dictionary; to write it to disk:

```
import json

with open("Well_A_results.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2, default=str)
```

6.5 Plot output curves and profiles

```
result.plot_output_curve()
```

The method plots wellhead pressure against total flow and flowing enthalpy using Matplotlib.

To add a profile variable to a Plotly figure:

```
fig = result.plot_profile_value(
    column="Press (Bara)",
    title="Pressure Profile"
)
fig.show()
```

The `column` argument must match a column name in `result.dataframes["wellbore_profile"]`. The current parser includes columns such as `Depth (m MD)`, `Press (Bara)`, `Temp(oC)`, `Mass Flow (kg/s)`, and XCO2.

7. Batch and sensitivity workflows

A robust batch workflow should:

- Load a validated base input model.
- Create a fresh output directory for each case.
- Modify only the intended parameter(s).
- Write each case to its own directory.
- Run the correct SwelFlo executable mode.
- Confirm that an output file was produced.
- Parse the output and store the relevant result metrics.
- Record case parameters, run status, and output paths in a summary table.

See `examples/test-run-loop.py` for a template. The parameter sweep in that example is deliberately explicit: users must confirm the correct SwelFlo field and unit convention before enabling it.

8. `geoPTS` utilities

The source also contains a `geoPTS` class for processing spinner/PTS survey data. Its current methods include:

- `add_well_track()` and `get_pos_depth()` for measured-depth calculation from a well track;
- `raw_data_smoothing()` for filtering abrupt changes in cable speed, pressure, temperature, and spinner readings;
- `mitch_algorithm()` for binning and aggregating measurements;
- `interp_pts_data()` for interpolation onto the generated well-depth array;
- `calc_slope()` for estimating spinner response against cable velocity;
- `calc_fluid_velocity()` for calculating fluid velocity from spinner response and slope data;
- `plot_slope()` and `plot_vfluid()` for visualization.

This functionality is included because it exists in the supplied source file, but its assumptions and algorithms should be validated against field procedures before operational use. It is not required for the basic SwelFlo input/output workflow.

9. Troubleshooting

```
| Symptom | Recommended check |
| `FileNotFoundError` when loading input/output | Confirm the full path and filename |
| Output parser returns empty DataFrames | Confirm that the output file is from a supported SwelFlo format/build and inspect th
| No sibling CSV tables appear | Check that `SWFplot###.csv` and `SWFoutcrv###.csv` exist beside the output text file |
| Silent run produces no output | Confirm executable path, working directory, input file, and whether that SwelFlo build suppor
| GUI opens but no result appears | Run the calculation in the GUI and confirm the output directory |
| Feedzone depth validation fails | Recalculate/set `model.length` and check depth units |
| Plot method reports missing data | Inspect available DataFrame keys and column names before plotting |
```

10. Engineering validation and limitations

PySwelFlo is a workflow automation and data-processing layer. It does not independently validate the physical assumptions, correlations, well geometry, fluid properties, or numerical convergence of a SwelFlo simulation.

Before using results for reservoir management or production forecasting:

- compare representative cases against SwelFlo GUI runs;
- verify input and output units;
- validate parsed values against the original output files;
- retain the input deck, executable version, and run logs for each case;
- review the wellbore model and assumptions with a qualified engineer.

11. Citation, license, and attribution

11.1 Citing PySwelFlo

If PySwelFlo contributes to academic research, engineering analysis, technical reports, presentations, publications, or derived software, please cite the repository and identify the version used.

Suggested citation:

> Canilandi, M. P. (2026). *PySwelFlo: Python tools for automating SwelFlo geothermal wellbore simulations* (Version 0.1.0) [Computer software]. GitHub. https://github.com/pierecnlnd/PySwelFlo

BibTeX:

```
@software{canilandi_pyswelflo_2026,
  author  = {Canilandi, Mesias Piere},
  title   = {PySwelFlo: Python Tools for Automating SwelFlo Geothermal Wellbore Simulations},
  year    = {2026},
  version = {0.1.0},
  url     = {https://github.com/pierecnlnd/PySwelFlo}
}
```

For reproducibility, cite the specific release tag or commit hash used in the work. GitHub can also generate a citation from the repository's `CITATION.cff` file using its **Cite this repository** feature.

Citation is requested as a scholarly and professional attribution practice. It does not add a separate legal condition to the MIT License.

11.2 License and third-party software

PySwelFlo is distributed under the MIT License (see the repository `LICENSE` file). SwelFlo is separate proprietary software and is not included in this project. Users must obtain their own valid SwelFlo installation and comply with its license terms.